SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: CSA1407 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. *(BL3)*

Assessment Tool Weightage: 10%

QUESTIONS: **Total Marks:50**

Annexure A

SCENARIO

BASED

TASK

Code of Conduct:

I Athavan K(192524036)certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student :Athavan K

RUBRICS FOR CALCULATION:

SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Scenario based assessment

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and

Parsing Table Conflict in LL(1)

Understanding: An LL(1) parser builds a predictive parsing table where each cell must hold exactly one production, so the parser can choose the right rule by looking just one token ahead without backtracking

How FIRST/FOLLOW are used: For a production $A \rightarrow a$, the entry $M[A, a]$ is filled for every terminal a in FIRST(a). If a can derive ε, the productions both qualify for the same (A, a) pair

Root cause of the conflict: This overlap happens when the grammar has:

- **Left recursion:** $A \rightarrow Aa \mid B$, which breaks top-down prediction entirely.
- **Common prefixes** two productions of A starting with the same symbols, making FIRST sets overlap (needs left factoring)

- Inherent ambiguity : The grammer allows more than one derivation for the same string.

Effect on parsing : LL (1) parsing is table-driven and deterministic by design — one lookahead token must map to one action. A cell with multiple entries forces the parser to guess, which it cannot do without backtracking, so parsing becomes non-deterministic and unreliable.

Decision : Eliminate left recursion (convert $A \rightarrow Aa \mid \beta$ into $A \rightarrow \beta A'$, $A' \rightarrow aA' \mid \varepsilon$), apply left factoring to production sharing a prefix, and re-drive FIRST/FOLLOW to confirm no overlaps remains This transformation the grammar into a valid, conflict-free LL(1) grammar.

2) Incorrect Parse Tree Generation (id + id − id)

Understanding : The parse tree's shape dicatetes evaluation order grouping is [id + id − id] left to right since + and − are conventionally left associative.

Key issue: If the grammar is written non-deterministically for associatively. eg: $E \rightarrow E + E$. $E - E \mid id$ and $id + (id - id)$ — giving different results. The grammar itself doesn't dictate which one to build, so the parser or parser generator) may construct the wrong tree.

How grammar shapes structure: Recursive productions determine associativity directly: left recursion ($E \rightarrow E + T$) forces left-to-right grouping. Since + and - are not naturally right associative, right recursion here produces incorrect results.

4) Decision: Rewrite the grammar with left recursion to enforce left associativity:

- $E \rightarrow E + T \mid E - T \mid T$
- $T \rightarrow id$

This removes structural ambiguity and guarantee every derivation produces the single, correctly grouped parse tree.

3) Invalid Expression Handling (a + * b)

Understanding: Lexical analysis only validates individual tokens a, +, *, b are all legal tokens) but syntax analysis validates whether the seq of tokens conforms to the grammar's structural rules.

How parsers detect it:

- Predictive (LL) parsing: the parser consults the parsing table for the current non-terminal and lookahead *; no valid entry exists, so it signals an error immediately.

- shift-reduce (LR) parsing: The parser has no valid shift action for * given the current stack state, and no reduction applies either so it reports an error at that point.

Without recovery, both approaches simply halt after the first error, which is unhelpful for identifying further problems in the same input.

Decision: Implement systematic error recovery

- Panic-mode recovery: discard tokens until a synchronizing token. often from FOLLOW(A)) is found, then resume parsing.

- Phrase-level recovery: Perform local fixes insert / delete / replace a token) to continue parsing.

- FOLLOW-set synchronization - use FOLLOW sets to decide safe resumption points.

This lets the compiles report multiple meaning errors in one pass instead of stopping at the first one.

4) Ambiguity in conditional statements.

Understanding: The grammar $S \rightarrow if\ \epsilon\ then\ S | if\ \epsilon\ then\ S\ else\ S | a$ allows nested if-statements without else to be ambiguous about which if an else attaches to - this is the classic dangling else problem.

Key issues :

Both productions share the prefix if E then S, so on encountering an else after a nested if... then ... if...then..., the parser cannot decide whether the Else binds to the inner or outer if. This creates a shift-reduce (or table) conflict and multiple valid parse trees for the same input.

Impact :

Ambiguity breaks determinism the parser has no unique way to decide, so different implementations could produce different program behavior.

Decision : Restructure the grammar to separate matched (fully closed. if-then-else) from unmatched statements.

- $S \rightarrow M \mid U$
- $M \rightarrow$ if ϵ then M else $M \mid a$
- $U \rightarrow$ if ϵ then $S \mid$ if ϵ then M else U

This forces every else to bind to the nearest unmatched if — the standard resolution used by most real-world programming languages.